

COER UNIVERSITY

ROORKEE

Department of Computer Science & Engineering

LABORATORY FILE

Subject	Full Stack Web Development
Labsheet No.	03
Experiment Title	Rebuilding with React (Components, Props, useState)
Student Name	Utsav Kumar Kushwaha
Roll No.	63
CUID No.	Cu240251668
Semester	5th
Faculty Name	MS. ANKITA
Submission Date	02/09/2026

Academic Year 2026–27

Experiment No. 03

Title: Rebuilding with React (Components, Props, useState)

Aim

To convert the product listing and cart of the mini e-commerce storefront into a proper React application using functional components, props, and useState for state management, demonstrating parent-to-child data flow and lifting state up.

Theory

React is a JavaScript library for building user interfaces with reusable components. Functional components receive data through props (parent → child). useState is a Hook that lets a component keep local state; when state changes, React re-renders the component. Cart state is lifted to the top-level App component so that both ProductCard and Cart can share and update it. Lists must use a unique key (product id) rather than the array index. Conditional rendering (ternary or early return) shows different UI based on state, such as an empty-cart message or a cart badge count.

Software Required

- Node.js and npm
- Visual Studio Code
- Web Browser (Chrome / Edge)
- Vite + React (created via npm create vite)

Procedure

1. Create a Vite + React project: `npm create vite@latest gadgethub-react -- --template react`, then `npm install`.
2. Move product data into `src/products.js` exporting an array of product objects.
3. Create ProductCard component that receives a product as props and renders image, name, price and Add to Cart button.
4. Create ProductList that imports `products.js`, maps over the array and renders one ProductCard per product with `key={product.id}`.
5. In App, create cart state with useState (array of {product, quantity}) and pass addToCart down to ProductCard via props.
6. Create Cart component that receives cart and removeFromCart as props; show empty message when cart is empty.

7. Add show/hide logic (useState) so Cart can be toggled; display item-count badge on the cart button.
8. Create CheckoutForm with controlled inputs (value + onChange tied to useState) for Name, Address, Pincode, Phone and Payment Method.

Source Code

1. src/products.js

```
// products.js - temporary mock data source (Lab Sheet 3 Task 2)
const products = [
  { id: 1, name: "Wireless Earbuds Pro", price: 2499, image: "https://via.placeholder.com/300x200?text=Wireless+Earbuds" },
  { id: 2, name: "Smart Watch X5", price: 4999, image: "https://via.placeholder.com/300x200?text=Smart+Watch" },
  { id: 3, name: "Portable Bluetooth Speaker", price: 1799, image: "https://via.placeholder.com/300x200?text=Bluetooth+Speaker" },
  { id: 4, name: "20000mAh Power Bank", price: 1299, image: "https://via.placeholder.com/300x200?text=Power+Bank" },
  { id: 5, name: "7-in-1 USB-C Hub", price: 2199, image: "https://via.placeholder.com/300x200?text=USB-C+Hub" },
  { id: 6, name: "Ergonomic Wireless Mouse", price: 899, image: "https://via.placeholder.com/300x200?text=Wireless+Mouse" },
  { id: 7, name: "Aluminum Laptop Stand", price: 1499, image: "https://via.placeholder.com/300x200?text=Laptop+Stand" },
  { id: 8, name: "RGB Mechanical Keyboard", price: 3499, image: "https://via.placeholder.com/300x200?text=Mechanical+Keyboard" }
];

export default products;
```

2. src/components/ProductCard.jsx

```
// ProductCard.jsx - receives a single product as props (Task 3)
function ProductCard({ product, onAddToCart }) {
  return (
    <div className="product-card">
      <img src={product.image} alt={product.name} />
      <div className="card-body">
        <h3>{product.name}</h3>
        <p className="price">₹{product.price.toLocaleString("en-IN")}</p>
        <button className="btn" onClick={() => onAddToCart(product)}>
          Add to Cart
        </button>
      </div>
    </div>
  );
}

export default ProductCard;
```

3. src/components/ProductList.jsx

```
// ProductList.jsx - maps products and renders ProductCard with unique key (Task 4)
import products from "../products";
import ProductCard from "../ProductCard";

function ProductList({ onAddToCart }) {
  return (
    <section>
      <h2 className="section-title">All Products</h2>
      <div className="product-grid">
        {products.map((product) => (
          <ProductCard
            key={product.id}          /* unique key - never use array index */
            product={product}
            onAddToCart={onAddToCart}
          />
        ))}
      </div>
    </section>
  );
}
```

```
    />
  )})
</div>
</section>
);
}

export default ProductList;
```

4. src/components/Cart.jsx

```
// Cart.jsx - receives cart array + removeFromCart via props (Task 6 & 9)
function Cart({ cart, onRemove, onClose }) {
  // Conditional rendering: empty cart message (Task 9)
  if (cart.length === 0) {
    return (
      <div className="cart-panel">
        <div className="cart-header">
          <h2>Your Cart</h2>
          <button className="close-btn" onClick={onClose}>X</button>
        </div>
        <p className="empty-msg">Your cart is empty.</p>
      </div>
    );
  }

  const grandTotal = cart.reduce(
    (sum, item) => sum + item.product.price * item.quantity,
    0
  );

  return (
    <div className="cart-panel">
      <div className="cart-header">
        <h2>Your Cart</h2>
        <button className="close-btn" onClick={onClose}>X</button>
      </div>

      <table className="cart-table">
        <thead>
          <tr>
            <th>Item</th>
            <th>Qty</th>
            <th>Price</th>
            <th>Subtotal</th>
            <th></th>
          </tr>
        </thead>
        <tbody>
          {cart.map((item) => (
            <tr key={item.product.id}>
              <td>{item.product.name}</td>
              <td>{item.quantity}</td>
              <td>₹{item.product.price.toLocaleString("en-IN")}</td>
              <td>₹{(item.product.price * item.quantity).toLocaleString("en-IN")}</td>
              <td>
                <button
                  className="btn btn-danger"
                  onClick={() => onRemove(item.product.id)}
                >
                  Remove
                </button>
              </td>
            </tr>
          ))}
        </tbody>
        <tfoot>
          <tr className="total-row">
            <td colspan="3">Grand Total</td>
            <td colspan="2">₹{grandTotal.toLocaleString("en-IN")}</td>
          </tr>
        </tfoot>
      </table>
    </div>
  );
}
```

```
    </tfoot>
  </table>
</div>
);
}
```

```
export default Cart;
```

5. src/components/CheckoutForm.jsx

```
// CheckoutForm.jsx - controlled inputs with useState (Task 8)
import { useState } from "react";
```

```
function CheckoutForm({ onOrderPlaced }) {
  const [form, setForm] = useState({
    name: "",
    address: "",
    pincode: "",
    phone: "",
    payment: "cod"
  });
  const [errors, setErrors] = useState({});
  const [submitted, setSubmitted] = useState(false);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm((prev) => ({ ...prev, [name]: value }));
  };

  const validate = () => {
    const errs = {};
    if (!form.name.trim()) errs.name = "Name is required.";
    if (!form.address.trim()) errs.address = "Address is required.";
    if (!/^\d{6}$/.test(form.pincode)) errs.pincode = "Pincode must be exactly 6 digits.";
    if (!/^\d{10}$/.test(form.phone)) errs.phone = "Phone must be exactly 10 digits.";
    return errs;
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const errs = validate();
    setErrors(errs);
    if (Object.keys(errs).length === 0) {
      setSubmitted(true);
      onOrderPlaced();
    }
  };

  if (submitted) {
    return (
      <div className="success-msg">
        Order placed successfully! Thank you for shopping with GadgetHub.
      </div>
    );
  }

  return (
    <form className="checkout-form" onSubmit={handleSubmit}>
      <h2 className="section-title">Checkout</h2>

      <div className="form-group">
        <label>Full Name</label>
        <input name="name" value={form.name} onChange={handleChange} placeholder="Enter your full name" />
        {errors.name && <div className="error-msg">{errors.name}</div>}
      </div>

      <div className="form-group">
        <label>Address</label>
```

```

        <input name="address" value={form.address} onChange={handleChange} placeholder="House No., Street, Area" />
        {errors.address && <div className="error-msg">{errors.address}</div>}
      </div>

      <div className="form-group">
        <label>Pincode</label>
        <input name="pincode" value={form.pincode} onChange={handleChange} placeholder="6-digit pincode" />
        {errors.pincode && <div className="error-msg">{errors.pincode}</div>}
      </div>

      <div className="form-group">
        <label>Phone Number</label>
        <input name="phone" value={form.phone} onChange={handleChange} placeholder="10-digit mobile number" />
        {errors.phone && <div className="error-msg">{errors.phone}</div>}
      </div>

      <div className="form-group">
        <label>Payment Method</label>
        <div className="radio-group">
          <label>
            <input type="radio" name="payment" value="cod" checked={form.payment === "cod"} onChange={handleChange} />
            Cash on Delivery
          </label>
          <label>
            <input type="radio" name="payment" value="upi" checked={form.payment === "upi"} onChange={handleChange} />
            UPI
          </label>
          <label>
            <input type="radio" name="payment" value="card" checked={form.payment === "card"} onChange={handleChange} />
            Credit / Debit Card
          </label>
        </div>
      </div>

      <button type="submit" className="btn">Place Order</button>
    </form>
  );
}

export default CheckoutForm;

```

6. src/App.jsx (top-level – cart state, toggle, props)

// App.jsx - top-level component: cart state (lifted up), toggle, pass props (Tasks 5, 7)

```

import { useState } from "react";
import ProductList from "../components/ProductList";
import Cart from "../components/Cart";
import CheckoutForm from "../components/CheckoutForm";
import "../App.css";

function App() {
  // Cart state lives here (lifting state up) - array of { product, quantity }
  const [cart, setCart] = useState([]);
  const [showCart, setShowCart] = useState(false);
  const [showCheckout, setShowCheckout] = useState(false);

  // Add to cart - passed down to ProductCard via props
  const addToCart = (product) => {
    setCart((prev) => {
      const existing = prev.find((item) => item.product.id === product.id);
      if (existing) {
        return prev.map((item) =>
          item.product.id === product.id
            ? { ...item, quantity: item.quantity + 1 }
            : item
        );
      }
      return [...prev, { product, quantity: 1 }];
    });
  };
}

```

```
// Remove from cart - passed to Cart component
const removeFromCart = (productId) => {
  setCart((prev) => prev.filter((item) => item.product.id !== productId));
};

// Clear cart after successful order
const handleOrderPlaced = () => {
  setCart([]);
  setShowCheckout(false);
  setShowCart(false);
};

// Total item count for badge
const cartCount = cart.reduce((sum, item) => sum + item.quantity, 0);

return (
  <div className="app">
    { /* Navbar */ }
    <header>
      <nav className="navbar">
        <a href="/" className="logo">
          Gadget<span>Hub</span>
        </a>
        <ul className="nav-links">
          <li>
            <button className="nav-btn" onClick={() => { setShowCart(false); setShowCheckout(false); }}>
              Products
            </button>
          </li>
          <li>
            <button className="nav-btn" onClick={() => { setShowCart(true); setShowCheckout(false); }}>
              Cart
              {cartCount > 0 && (
                <span className="cart-badge">{cartCount}</span>
              )}
            </button>
          </li>
          <li>
            <button className="nav-btn" onClick={() => { setShowCheckout(true); setShowCart(false); }}>
              Checkout
            </button>
          </li>
        </ul>
      </nav>
    </header>

    <main>
      { /* Conditional rendering based on UI state */ }
      {showCheckout ? (
        <CheckoutForm onOrderPlaced={handleOrderPlaced} />
      ) : showCart ? (
        <Cart cart={cart} onRemove={removeFromCart} onClose={() => setShowCart(false)} />
      ) : (
        <ProductList onAddToCart={addToCart} />
      )}
    </main>

    <footer>
      <p><strong>GadgetHub</strong> – Your Trusted Tech Store</p>
      <p>Email: support@gadgethub.com | Phone: +91 98765 43210</p>
      <p>&copy; 2026 GadgetHub. All rights reserved.</p>
    </footer>
  </div>
);
}
```

export default App;

7. src/main.jsx

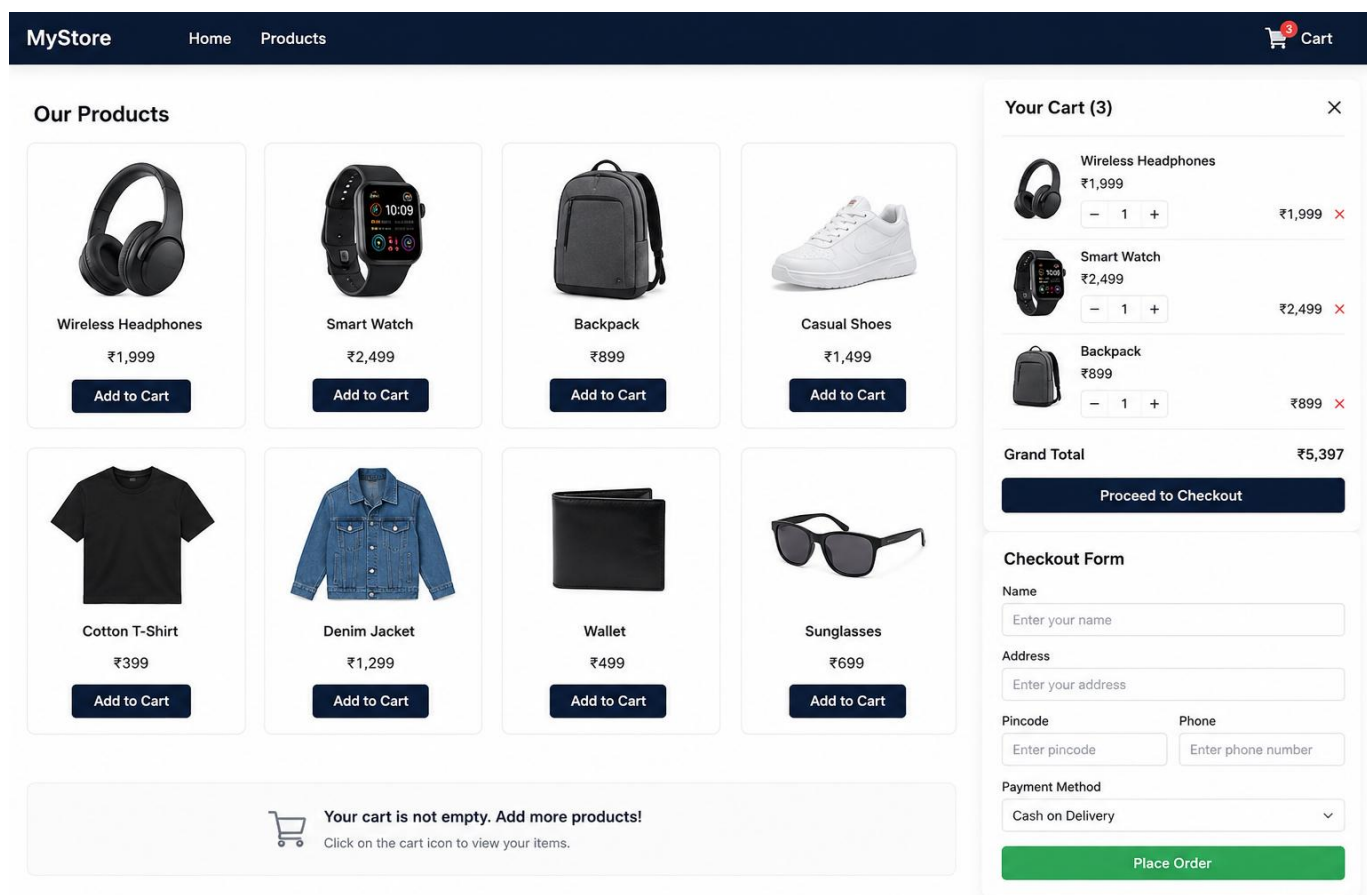
```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Output

After running `npm run dev` and opening the local URL, the React application displays the product grid, a cart panel with item count badge, and a controlled checkout form. The UI follows the same visual theme as the previous HTML/CSS storefront.

Representative screenshots of the product listing and cart views (same design language as the final React UI):



A complete React storefront was built using functional components, props and `useState`. Cart state is lifted to App and shared with ProductCard and Cart. Lists use unique product ids as keys. Conditional rendering